

Optimizing Deployment of Irregular Sparse Deep Learning Workloads through ILP-Based Scheduling for Low-Latency Inference



Changxin Li, Professor Sanmukh Kuppannagari
cxl1492@case.edu, sxk1942@case.edu

Case Western Reserve University – Department of Computer and Data Sciences



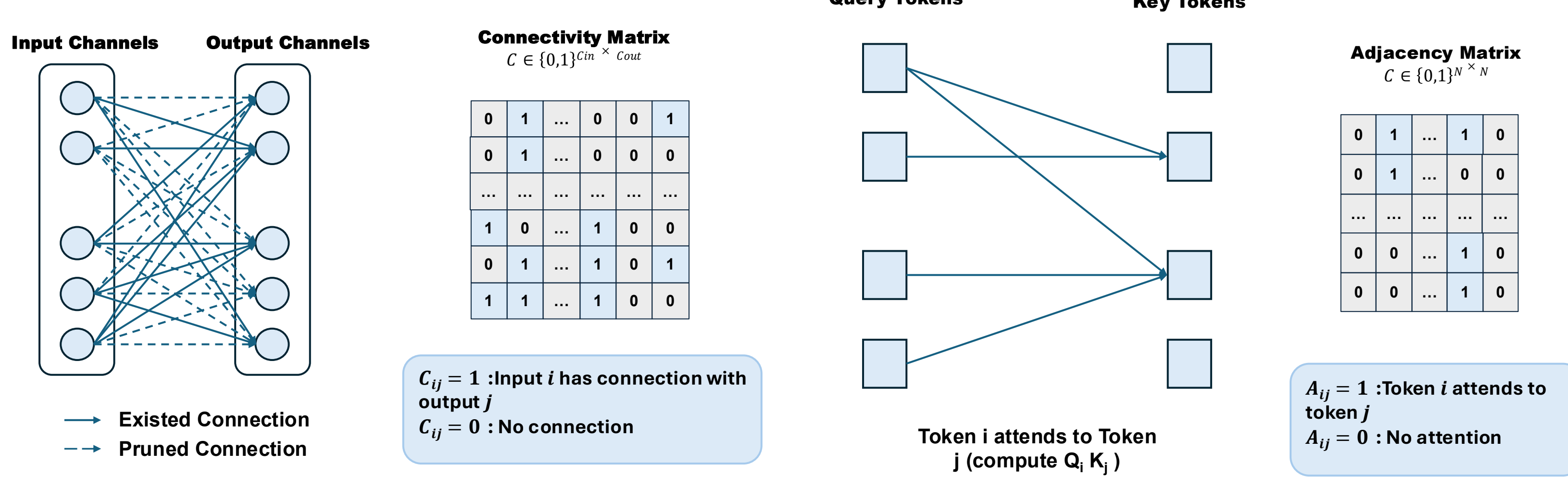
Introduction

Irregular sparse workloads are increasingly used in CNNs and Vision Transformers to reduce computation. In CNNs, unstructured group convolutions introduce irregular input-output channel connections. In Vision Transformers, sparse attention produces token-interaction blocks with different sizes and densities. Although these sparse patterns reduce theoretical FLOPs, existing GPU kernels often fail to fully exploit them because they rely on dense execution, fixed tiling, or regular group structures.

In this work, we propose a unified ILP-based scheduling framework for efficient sparse workload deployment. Our method represents sparse computation as matrix structures, applies reordering to improve locality, and uses offline profiling to collect latency and resource costs for candidate configurations. An ILP solver then selects hardware-feasible execution schedules under GPU resource constraints. By converting irregular sparsity into optimized GPU execution plans, our framework improves inference latency while preserving model accuracy.

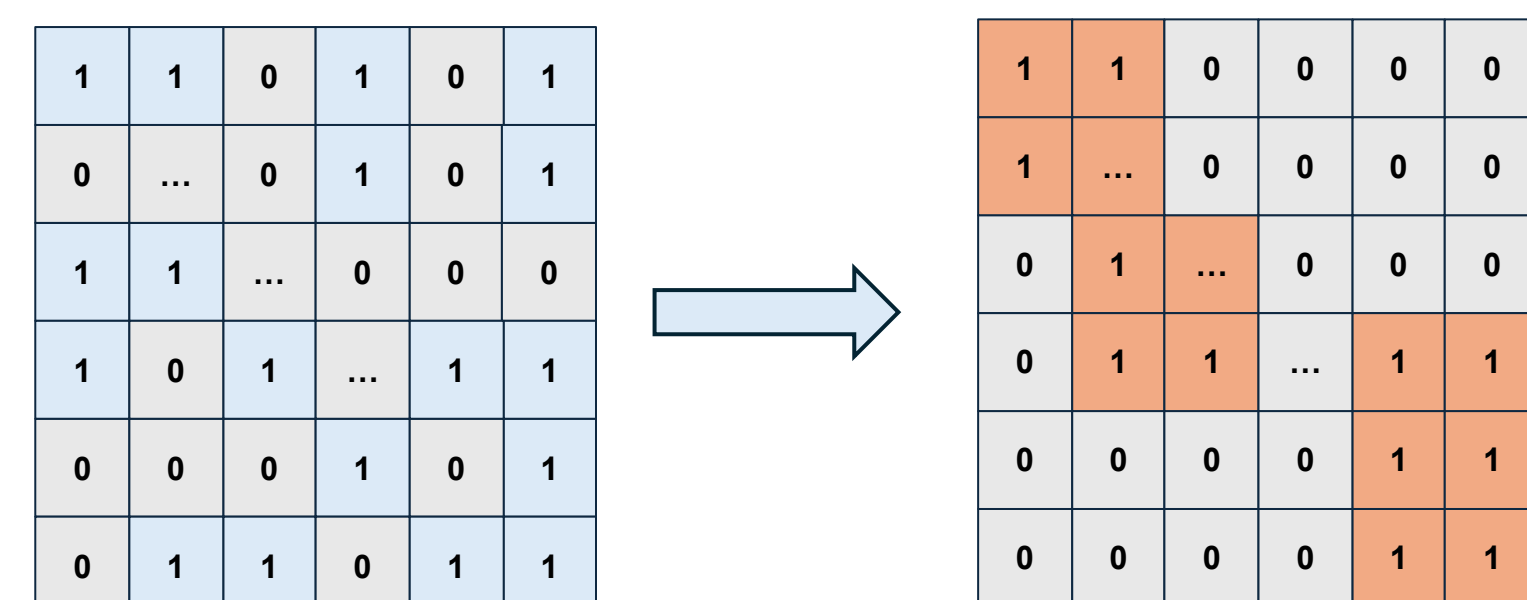
Optimization

(1) Represent Sparse Structure



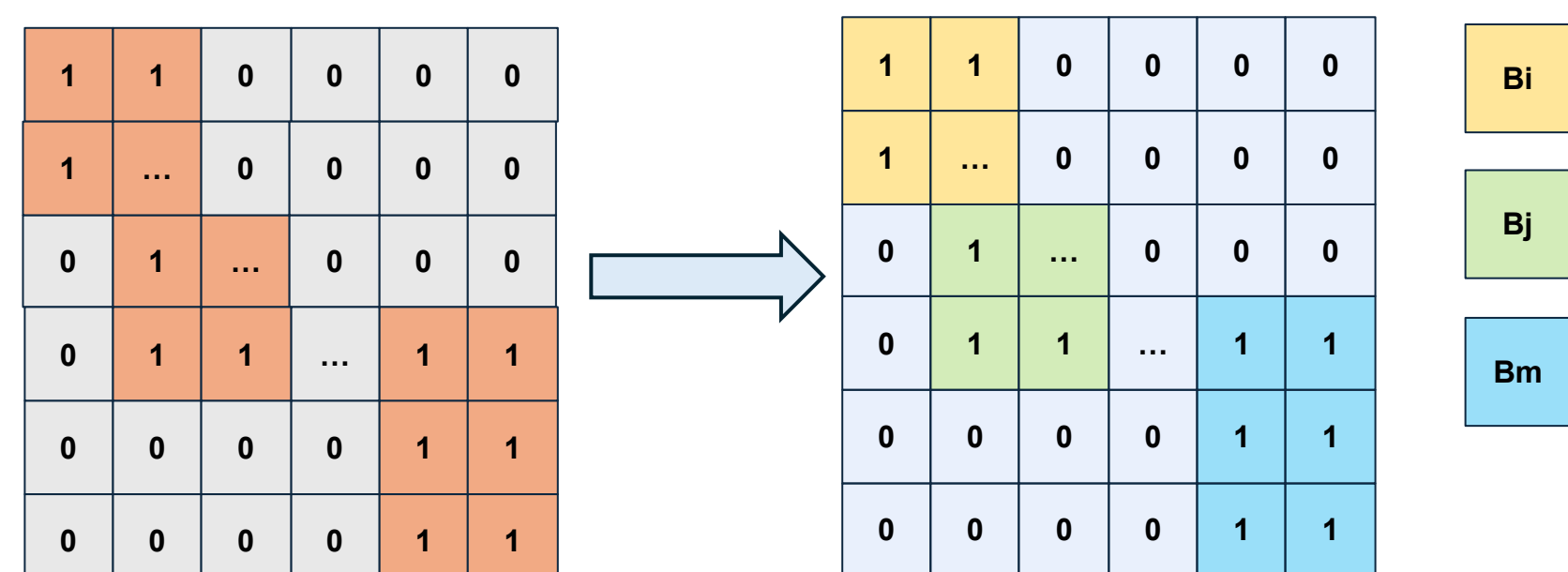
(2) Reorder for Locality

We apply reordering algorithms to rearrange matrices. This clusters nonzero entries into denser regions, improving locality and exposing GPU-friendly block structures.



(3) Extract Scheduling Units

Partition the reordered structure into smaller computation regions that can be independently analyzed and scheduled.



(4) Estimate Execution Cost

Sparse Unit (u)	Represents	Candidate Config (c) (Execution Option)	Latency $L_{u,c}$ (ms)	GPU Resource Usage $R_{u,c}$ (e.g., SMEM / Regs)	Occupancy (%)	Feasible (Under Constraints)
u_1	CNN Connection Group (e.g., group of channel connections)	c_1 : Partition size = 64	0.183	16 KB / 28 Regs	87.5	✓
		c_2 : Partition size = 128	0.121	48 KB / 40 Regs	75.0	✓
		c_3 : Partition size = 256	0.145	96 KB / 64 Regs	50.0	✓
u_2	Irregular Block (e.g., irregular sparse computation block)	c_1 : Config = A (coarse tiling)	0.118	40 KB / 36 Regs	78.1	✓
		c_2 : Config = B (balanced tiling)	0.086	72 KB / 52 Regs	62.3	✓
		c_3 : Config = C (fine tiling)	0.139	128 KB / 72 Regs	41.2	✓

Use profiling or lightweight cost models to estimate latency and hardware resource usage for candidate execution configurations.

(5) Optimize with ILP

Unified ILP Scheduler

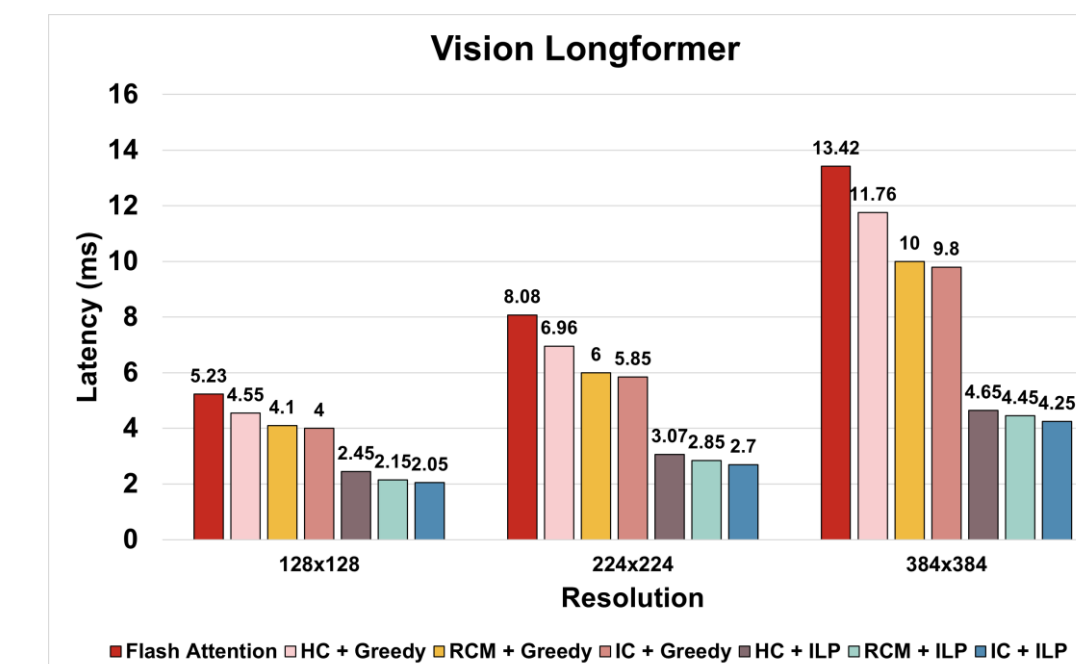
ILP Formulation Overview	
Input:	sparse units + candidate configs
Goal:	minimize latency
Constraint:	GPU resource limits
Output:	optimized execution plan

ILP selects the best execution configuration for each sparse unit to minimize latency while satisfying GPU resource constraints.

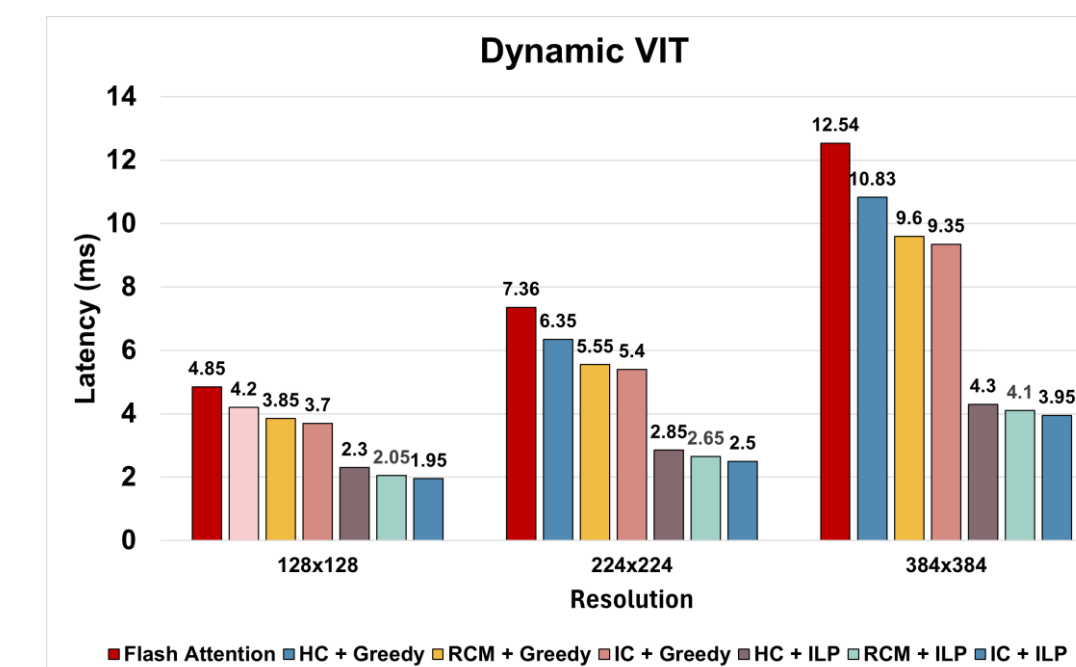
ILP selects the best configuration for each sparse unit under GPU constraints.

Results

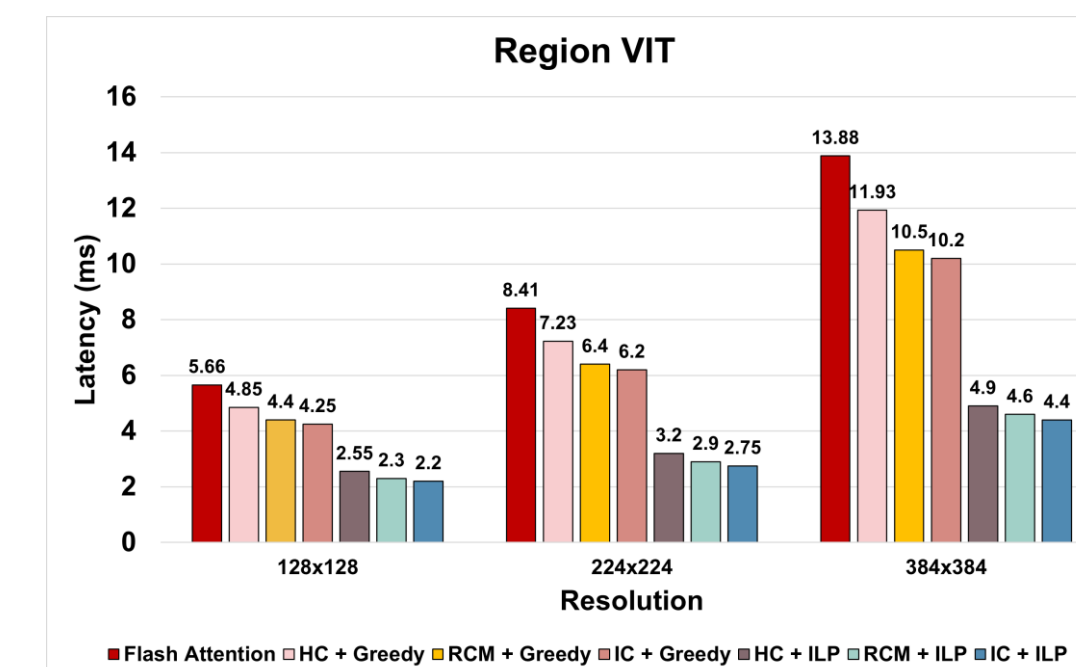
A. Vision Transformer



- IC+ILP speedup increases from $2.55\times$ at 128×128 to $3.16\times$ at 384×384 .
- Greedy scheduling reaches only $1.37\times$ speedup at 384×384 , while IC+ILP reaches $3.16\times$.
- ILP reduces latency by $2.31\times$ over IC+Greedy at 384×384 .



- DynamicViT reaches the highest peak speedup: $3.17\times$ at 384×384 .
- Speedup improves from $2.49\times$ at 128×128 to $3.17\times$ at 384×384 .
- ILP gives a $2.37\times$ latency reduction over IC+ at 384×384 .



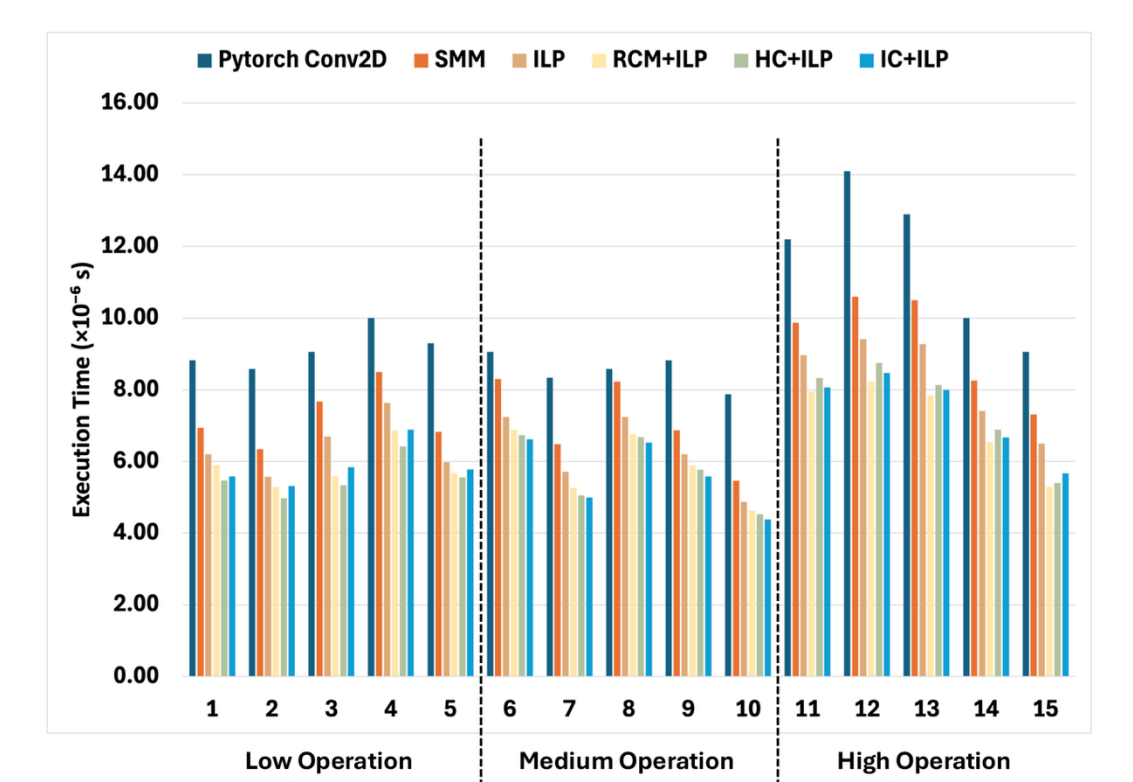
- RegionViT improves from $2.57\times$ speedup at 128×128 to $3.15\times$ at 384×384 .
- IC+Greedy gives only $1.36\times$ speedup at 384×384 , while IC+ILP reaches $3.15\times$.
- ILP reduces latency by $2.32\times$ over IC+Greedy at 384×384 .

Model	HC+ILP	RCM+ILP	IC+ILP
Vision Longformer	$1.91\times$	$2.05\times$	$2.13\times$
RegionViT	$1.88\times$	$1.99\times$	$2.09\times$
DynamicViT	$1.86\times$	$1.97\times$	$2.06\times$

Model	Params (M)	FLOPs (G)	Baseline Acc. (%)	Ours (%)
Vision Longformer	55.7	13.4	83.7	83.7
RegionViT	72.7	13.0	83.2	83.2
DynamicViT	57.1	9.6	83.9	83.9

B. CNN

- ILP-based methods still consistently reduce execution time across layers.
- IC+ILP performs best, reaching up to $1.75\times$ speedup over PyTorch Conv2D.



References

- [1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," Advances in neural information processing systems, vol. 30, 2017.
- [2] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. R'e, "Flashattention: Fast and memory-efficient exact attention with io-awareness," 2022. [Online]. Available: <https://arxiv.org/abs/2205.14135>
- [3] M. Shan, D. Gurevin, J. Nye, C. Ding, and O. Khan, "Mergepath-spmm: Parallel sparse matrix-matrix algorithm for graph neural network acceleration," in 2023 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS), 2023, pp. 145–156.
- [4] T. Dao, D. Fu, S. Ermon, A. Rudra, and C. R'e, "Flashattention: Fast and memory-efficient exact attention with io-awareness," Advances in neural information processing systems, vol. 35, pp. 16 344–16 359, 2022.
- [5] T. Dao, "Flashattention-2: Faster attention with better parallelism and work partitioning, 2023," URL <https://arxiv.org/abs/2307.08691>, vol. 5, no. 10, 2023.